

Snap – del 2: skalning, chatt och push-notiser

Implementera arkitekturen i `architecture.png`.

Läget

- **Backend:** auth, vänner och snaps finns. Websocket-notiser via socket.io, bara från server till klient. Ingen chatt och inga push-tokens. `docker-compose.yml` innehåller bara Postgres.
- **Frontend:** alla skärmar finns men kör mot mocken (`lib/api.ts`). `lib/api.real.ts` är klar. Chattskrmen är bara ett skal.

Regler

- Ändra inte befintliga endpoints eller websocket-events.
- Nya endpoints, events och tabeller föreslås i gruppchatten innan ni bygger dem, och dokumenteras.
- All kommunikation med backend går genom `lib/`.

Uppgifter

Gör dem uppifrån och ner.

1. **Koppla frontend till riktig backend.** Byt från mocken. Register, login, vänner och förfrågningar ska fungera, och backendens felmeddelanden ska visas.
2. **Docker Compose.** `docker compose up` startar allt. Tjänsterna ligger i ett eget nätverk, och enda vägen in är Nginx, även för websocket. Inga hemligheter i repot. S3 och Cloudflare ingår inte.
3. **Flera instanser.** Minst två backend-instanser bakom Nginx med round-robin. Antalet ska gå att ändra utan kodändringar.
4. **Chatt mellan vänner, enligt arkitekturen.** Meddelanden kommer fram i realtid även när vännerna är anslutna till olika instanser. Historiken sparas. Man kan bara chatta med vänner där `mutual: true`. De befintliga websocket-eventen ska också nå rätt användare, oavsett instans.
5. **Push-notiser.** Be om tillåtelse och spara push-token i databasen. Den som är offline får en notis när ett meddelande kommer. Läs Expos dokumentation för SDK 54, eftersom Expo Go inte stödjer push på samma sätt på iOS och Android.

Om tid finns

6. **Ta emot snaps.** Backend och att skicka snaps finns redan. Visa inkommande snaps, låt dem öppnas en gång och meddela avsändaren om mottagaren tar en skärmdump.
7. **Sharda databasen.** En DB Router framför Postgres A och B, som i bilden.

Se upp: socket.io bakom round-robin

socket.io börjar med HTTP long-polling och uppgraderar sedan till websocket. Sessionen finns bara i minnet hos den instans som tog emot det första anropet. Med round-robin hamnar nästa anrop ofta på en annan instans, som svarar `400 Session ID unknown`. Med en instans märks ingenting, och med två fungerar det bara ibland.

Välj en av lösningarna och motivera valet:

1. **Bara websocket:** sätt `transports: ['websocket']` i klienten. Nackdel: det finns ingen reserv om ett nätverk blockerar websockets.
2. **Sticky sessions:** använd `ip_hash` i Nginx. Nackdel: samma IP går alltid till samma instans, så två simulatorer på samma dator hamnar på samma instans.

Ni ska kunna förklara

- Hur ett meddelande når en användare som är ansluten till en annan instans.
- Hur ni avgör att någon är offline.
- Varför bara Nginx är nåbar utifrån.

Kom igång: läs `snap/README.md` och `backend/.env.example`.